

O analiză experimentală a algoritmilor de sortare uzuali. Când Sleep Sort câștigă.

Cristian Meriacri^{*,a,1}

^a Universitatea de Vest din Timișoara, Facultatea de Informatică, Str. Pârvan 4, Timișoara 300223, Romania - cristian.meriacrioo@e-uvv.ro

Acest articol a fost redactat în perioada Februarie-Mai 2026.

Abstract

Această lucrare prezintă o analiză experimentală a performanței algoritmilor de sortare, comparând soluții clasice precum Bubble Sort, Insertion Sort, Quick Sort, Merge Sort și implementarea de referință din biblioteca standard C++, std::sort. Un element central și neconvențional al studiului este includerea algoritmului Sleep Sort, testat pentru a identifica limitele practice ale eficienței bazate pe multi-threading.

Metodologia a presupus implementarea în C++ folosind compilatorul GCC și testarea pe diverse seturi de date, cu dimensiuni cuprinse între 10 și 2.000.000 de elemente și cinci distribuții de valori diferite. Principalul parametru monitorizat a fost timpul de execuție, măsurat în timpul real de pe ceasul de perete folosind librăria standard inclusă chrono, pentru a evalua scalabilitatea și performanța fiecărui algoritm.

Rezultatele obținute au evidențiat diferențe semnificative în performanță, cu algoritmi precum Quick Sort și Merge Sort demonstrând o eficiență superioară în comparație cu soluțiile mai simple. Totuși, deși majoritatea rezultatelor au fost pe măsura așteptărilor, bazat pe ce știm despre algoritmii de sortare, Sleep Sort a prezentat un comportament neașteptat, depășind performanța algoritmului std::sort din librăria C++ în anumite condiții.

Lucrarea demonstrează astfel că eficiența unui algoritm este strâns legată de natura distribuției datelor și de dimensiunea acestora, evidențiind importanța alegerii algoritmului potrivit pentru fiecare situație specifică.

Keywords: Algoritmi, Sortare, Bubble Sort, Insertion Sort, Quick Sort, Merge Sort, Sleep Sort, std::sort, C++, Performanță, multi-threading, chrono

■ CONTENTS

1	Introducere	2
1.1	Declarație de originalitate	2
2	Date de Intrare	2
2.1	Generalități	2
2.2	Tipuri de date	2
	Aleatoriu • Sortat Crescător • Sortat Descrescător • Aproape Sortat • Plat	
3	Algoritmi de sortare	2
3.1	Bubble Sort	2
3.2	Insertion Sort	2
3.3	Merge Sort	3
3.4	Quick Sort	3
3.5	std::sort	3
3.6	Sleep Sort	4
	Determinarea timpului de <i>somn</i> • Problema elementelor duplicate • Complexitatea algoritmului și influența sistemului de operare și hardware-ului • Comparația cu Counting și Radix Sort • Numere negative și numere raționale	
3.7	Ipoteze și așteptări	5
4	Rezultate	5
4.1	Bubble Sort - Analiză	6
4.2	Insertion Sort - Analiză	6
4.3	Merge Sort - Analiză	6
4.4	Quick Sort - Analiză	7
4.5	std::sort - Analiză	7
4.6	Sleep Sort - Analiză	7

5	Comparație cu literatura actuală	7
6	Concluzie	8
	Github Repository	8

1. INTRODUCERE

Performanța unui program depinde adesea mai mult de alegerea algoritmului decât de orice altă optimizare. În contextul sortării, una dintre operațiile cele mai frecvente în software, această alegere poate face diferența pe date reale, care rareori sunt perfect distribuite.

Această lucrare prezintă o analiză experimentală a șase algoritmi de sortare pe distribuții diverse, cu accent pe comportamentul lor în scenarii nefavorabile. Un element distinctiv este includerea Sleep Sort, un algoritm neconvențional bazat pe concurență, pentru a investiga dacă poate fi competitiv în condiții specifice.

Lucrarea este organizată astfel: Secțiunea ?? descrie algoritmi, Secțiunea ?? prezintă configurația experimentală, Secțiunea ?? analizează rezultatele, iar Secțiunea ?? sintetizează concluziile.

1.1. Declarație de originalitate

Implementarea Sleep Sort propusă este propria mea contribuție: am observat că varianta clasică creează câte un thread per element, inefficient pe date cu duplicate, și am rezolvat asta grupând elementele identice printr-un `std::map` pe același fir de execuție.

2. DATE DE INTRARE

2.1. Generalități

Această secțiune oferă o prezentare generală a datelor folosite în această lucrare, evidențiind principiile de funcționare și complexitatea lor teoretică. Algoritmii de sortare (mai puțin Sleep Sort) pot sorta orice tip de date, atâta timp cât există o relație de ordine definită între ele.

Pe parcursul acestei lucrări, toate datele de intrare sunt de tip integer.

2.2. Tipuri de date

Fie $n \in \mathbb{N}$ dimensiunea șirului și $A = (a_1, a_2, \dots, a_n)$ un șir de numere întregi, $a_i \in \mathbb{Z}$. Algoritmii au fost testați pe cinci distribuții de date, alese pentru a acoperi scenarii reprezentative din practică.

2.2.1. Aleatoriu

A este generat aleatoriu, unde fiecare $a_i \sim \mathcal{U}(1, 10^6)$, independent și uniform distribuit. Aceasta reprezintă cazul general, fără nicio structură prealabilă în date.

2.2.2. Sortat Crescător

A este un șir sortat crescător, unde $a_i \leq a_{i+1}$ pentru orice $i = 1, \dots, n-1$. Reprezintă cel mai favorabil scenariu pentru algoritmii care exploatează ordinea existentă.

2.2.3. Sortat Descrescător

A este un șir sortat descrescător, unde $a_i \geq a_{i+1}$ pentru orice $i = 1, \dots, n-1$. Reprezintă cazul defavorabil pentru majoritatea algoritmilor bazați pe comparații.

2.2.4. Aproape Sortat

A este inițial sortat crescător, după care $[0.01 \cdot n]$ perechi de elemente sunt interschimbate aleatoriu. Astfel, aproximativ 1% din elemente sunt în afara poziției corecte, simulând date reale care au suferit modificări minore după o sortare anterioară.

2.2.5. Plat

A este generat aleatoriu, unde $a_i \in \{v_1, v_2, v_3, v_4, v_5\}$, un set fix de exact 5 valori distincte. Această distribuție testează comportamentul algoritmilor în prezența unui număr foarte mare de duplicate.

3. ALGORITMI DE SORTARE

Această secțiune oferă o prezentare generală a algoritmilor de sortare analizați în această lucrare, evidențiind principiile de funcționare și complexitatea lor teoretică.

3.1. Bubble Sort

Bubble Sort este cel mai simplu algoritm de sortare, care funcționează prin compararea elementelor adiacente și interschimbarea lor dacă sunt în ordine greșită. Acest proces se repetă până când întregul șir este sortat. Deși este un algoritm foarte inefficient, Bubble Sort este adesea folosit în scop educațional pentru a ilustra conceptele de bază ale sortării.

Origine și popularitate

Originea acestui algoritm este incertă [6], fiind menționat sub diferite denumiri în literatura de specialitate ca "Sorting by Exchange", "Sinking Sort", "Jump Down", "Sink Down" sau "Exchange Sort".

Knuth l-a inclus în The Art of Computer Programming [4], contribuind la consolidarea lui în literatura de specialitate, deși l-a criticat explicit.

Complexitatea [4] a acestui algoritm este $O(n^2)$ în varianta de bază, fără optimizări, și $O(n)$ în cel mai favorabil caz (când șirul este deja sortat) dacă este folosită o implementare optimizată. În medie, complexitatea este tot $O(n^2)$, ceea ce îl face inefficient pentru șiruri mari.

Totodată, Bubble sort e mai încet decât alți algoritmi de sortare cu o complexitate $O(n^2)$ precum Insertion Sort sau Selection Sort (cu unele excepții evidențiate mai târziu), deoarece necesită mai multe interschimbări [6] pentru a sorta șirul.

Complexitatea de spațiu a acestui algoritm este $O(1)$, deoarece sortarea se face in-place (fără a alocă structuri de date auxiliare proporționale cu n), fără a necesita spațiu suplimentar semnificativ.

```
template <typename T>
void bubbleSort(std::vector<T>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

Code 1. Implementarea Bubble Sort în C++.

După cum se poate observa în codul sursă 1, algoritmul parcurge șirul de la început până la sfârșit, comparând elementele adiacente și interschimbându-le dacă sunt în ordine greșită. Un flag numit swapped este folosit pentru a verifica dacă s-au făcut interschimbări în timpul unei treceri. Dacă nu s-a făcut nicio interschimbare, înseamnă că șirul este deja sortat și algoritmul se oprește, optimizând astfel cazul favorabil.

3.2. Insertion Sort

Insertion Sort funcționează prin construirea treptată a șirului sortat, inserând fiecare element în poziția corectă în raport cu cele deja sortate. Prima mențiune documentată îi aparține lui John Mauchly în 1946, deși algoritmul era probabil folosit informal înainte [4].

Algoritmul este eficient pentru șiruri mici sau aproape sortate, având o complexitate de $O(n)$ în cel mai favorabil caz și $O(n^2)$ în cazul mediu și defavorabil [4]. Complexitatea de spațiu este $O(1)$, deoarece sortarea se face in-place.

Knuth menționează că poziția de inserare poate fi găsită prin căutare binară [4], reducând numărul de comparații la $O(\log n)$, fără a îmbunătăți însă complexitatea totală. O variantă mai agresivă, Gap Insertion Sort, folosește goluri între elemente pentru a reduce și mutările, obținând $O(n \log n)$ în majoritatea cazurilor [7].

```
template <typename T>
void insertionSort(std::vector<T>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        T key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

Code 2. Implementarea Insertion Sort în C++.

Implementarea Insertion Sort prezentată în codul sursă 2 este o versiune clasică, fără optimizări ca căutarea binară pentru poziția de inserare menționată anterior.

3.3. Merge Sort

Merge Sort este un algoritm de sortare bazat pe paradigma divide et impera, care împarte șirul în subșiruri mai mici, le sortează recursiv și apoi le combină pentru a obține șirul sortat final. A fost inventat de John von Neumann în 1945 [4]. Eficiența sa este de $O(n \log n)$ în toate cazurile, fără surprize sau cazuri excepționale, ceea ce îl face potrivit pentru șiruri mari.

Un dezavantaj al acestui algoritm este complexitatea de spațiu, care este $O(n)$, deoarece necesită un spațiu auxiliar pentru a stoca subșirurile în timpul procesului de combinare. Totodată, natura recursivă a algoritmului necesită un spațiu suplimentar pentru stiva de apeluri, ceea ce îl face inefficient pentru șiruri foarte mici sau pentru sisteme cu resurse limitate.

```
template <typename T>
void merge(std::vector<T>& arr, int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    std::vector<T> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
}

template <typename T>
void mergeSort(std::vector<T>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}
```

Code 3. Implementarea Merge Sort în C++.

Implementarea prezentată în codul sursă 3 folosește două funcții: `merge`, care combină două subșiruri sortate într-unul singur, și `mergeSort`, care realizează divizarea recursivă a șirului și apelează funcția de combinare.

3.4. Quick Sort

Quick Sort este un algoritm de sortare eficient, bazat pe paradigma divide et impera, care a fost inventat de Tony Hoare în 1959 și publicat în 1961 [1]. Algoritmul funcționează prin alegerea unui element pivot din șir și împărțirea celorlalte elemente în două subșiruri: cele mai mici decât pivotul și cele mai mari decât pivotul. Acest proces se repetă recursiv pentru fiecare subșir până când întregul șir este sortat.

Quick Sort, în ciuda numelui său, nu garantează performanță superioară în toate cazurile [1, 4].¹ O problemă cunoscută a acestui algoritm este că performanța sa poate degrada la $O(n^2)$ în cazul în care pivotul ales este întotdeauna cel mai mic sau cel mai mare element, cum ar fi atunci când șirul este deja sortat sau sortat descrescător. Cu toate acestea, cu o alegere bună a pivotului (de exemplu, folosind metoda medianei), complexitatea medie și cea mai bună rămân $O(n \log n)$ [4]. O concepție comună este că Quick Sort este întotdeauna mai rapid decât Merge Sort sau Bubble Sort, dar acest lucru nu este întotdeauna adevărat, deoarece Quick Sort are nevoie de mai mult overhead pentru gestionarea recursiei și poate fi inefficient pe șiruri mici sau aproape sortate, unde Insertion Sort sau Bubble Sort pot performa mai bine [4]. Aceeași problemă o suferă și Merge Sort. Complexitatea de spațiu a Quick Sort este $O(\log n)$ în medie, datorită stivei de apeluri recursive, dar poate ajunge la $O(n)$ în cel mai defavorabil caz.

O optimizare separată, numită Algoritmul Flagului Olandez [2], propusă de Dijkstra, modifică partiționarea pentru a gestiona eficient elementele duplicate — în loc de două zone (mai mic și mai mare decât pivotul), șirul este împărțit în trei: mai mic, egal și mai mare. Aceasta reduce semnificativ numărul de operații pe distribuții cu multe duplicate, cum ar fi distribuția plată din această lucrare.

```
template <typename T>
int partition(std::vector<T>& arr, int low, int high) {
    T pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

template <typename T>
void quickSort(std::vector<T>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Code 4. Implementarea Quick Sort în C++.

Pentru a ilustra consecințele alegerii pivotului, implementarea prezentată în codul sursă 4 folosește ultimul element ca pivot, ceea ce poate duce la performanță slabă pe șiruri deja sortate sau sortate descrescător. Pentru a evita acest lucru, se pot implementa strategii de alegere a pivotului, cum ar fi alegerea unui element aleatoriu sau folosirea medianei a trei elemente.

3.5. std::sort

`std::sort` este o funcție de sortare din biblioteca standard C++ care oferă o implementare optimizată, adaptată la dimensiunea și structura datelor. De obicei, `std::sort` utilizează o combinație de Quick Sort, Heap Sort și Insertion Sort, denumită Introsort [3, 15].

Implementarea specifică poate varia între diferite compilatoare și platforme. În compilatorul GCC, `std::sort` folosește o variantă a

¹ Observație atribuită cursului de Algoritmi și Structuri de Date, Universitatea de Vest din Timișoara, de Prof. Dr. D. Zaharie.

Introsort care începe cu Quick Sort și comută la Heap Sort dacă adâncimea recursivă depășește un prag specificat, prevenind astfel degradarea la $O(n^2)$ pe șiruri sortate sau aproape sortate. Pentru subșiruri foarte mici, algoritmul folosește Insertion Sort, mai eficient în acest context.

Complexitatea `std::sort` este $O(n \log n)$ în toate cazurile [3], inclusiv cel defavorabil. Complexitatea de spațiu este $O(\log n)$, datorită stivei de recursivitate. Algoritmul nu este stabil — pentru stabilitate, biblioteca standard oferă `std::stable_sort`.

Codul sursă al `std::sort` nu este unic, fiind adaptat diferitelor platforme și compilatoare. O implementare de referință a Introsort poate fi găsită în literatura de specialitate [3], iar implementarea din GCC este disponibilă în fișierul `bits/stl_algo.h` [16].

```
template<typename _RandomAccessIterator, typename _Compare>
inline void
__sort(_RandomAccessIterator __first, _RandomAccessIterator
    < __last,
        _Compare __comp)
{
    if (__first != __last)
    {
        std::__introsort_loop(__first, __last,
                               std::__lg(__last - __first) * 2,
                               __comp);
        std::__final_insertion_sort(__first, __last, __comp);
    }
}
```

Code 5. Funcția "caller" pentru `std::sort` în `stl_algo.h`.

În această lucrare, `std::sort` are rolul de referință pentru evaluarea algoritmilor implementați manual, oferind un punct de reper reprezentând o soluție optimizată și bine testată în scenarii diverse de date.

O misconcepție foarte răspândită printre fanii și puriștii limbajului C++ este că `std::sort` este cel mai rapid algoritm de sortare posibil, dar acest lucru nu este întotdeauna adevărat.

Deși `std::sort` este extrem de optimizat și oferă performanță excelentă în majoritatea cazurilor, există algoritmi mai eficienți pentru date reale.

Un exemplu este Timsort [5], un algoritm hibrid care combină Merge Sort și Insertion Sort, optimizat pentru date care conțin secvențe deja sortate. Timsort este folosit în Python, Java, Rust și multe alte limbaje populare, iar în scenarii uzuale poate depăși performanța `std::sort`, în special pe șiruri aproape sortate sau cu multe duplicate.

3.6. Sleep Sort

Această secțiune va fi cea mai detaliată, deoarece toți ceilalți algoritmi au fost discutați de mii de ori în literatura de specialitate până când a *adoarmit* fiecare student la algoritmi și structuri de date, iar Sleep Sort nu are literatură de specialitate. Nu doresc să reinventez roata algoritmilor de sortare, mă interesează ce se întâmplă când roata *adoarme*.

Sleep sort este un algoritm creat de o persoană necunoscută pe platforma 4chan [9][14][8] [13][12][11] în 2011. Postarea [8] de pe 4chan a atras multă atenție asupra sa, având sute de comentarii, dar pentru oricine dorește să *doarmă* bine noaptea nu e recomandat de citit mai mult de primele mesaje, considerând natura nemoderată a platformei.

Acest algoritm funcționează prin crearea unui thread [20] pentru fiecare element din șir, unde fiecare thread "*doarme*" [19] pentru o perioadă de timp proporțională cu valoarea elementului său. După ce se trezește, thread-ul tipărește valoarea sa, rezultând un șir sortat în ordine crescătoare.

Deși e o idee foarte simplă, această metodă de sortare deschide ușa unor întrebări interesante:

- Cum determinăm timpul de *somn* corect pentru fiecare thread?
- Ce se întâmplă când thread-urile se trezesc în același timp, cum ar fi în cazul valorilor duplicate?

- Cum am putea măsura complexitatea acestui algoritm? Cum diferă performanța în funcție de sistemul de operare și hardware-ul folosit?
- Cum se compară performanța Sleep Sort cu alți algoritmi bazați pe altceva decât comparații, cum ar fi Counting Sort sau Radix Sort, în special pe șiruri cu multe duplicate sau cu o gamă limitată de valori?
- Ce face Sleep Sort cu numere negative sau numere raționale, având în vedere că timpul nu poate fi negativ (la momentul scrierii acestei lucrări) și nu poate fi proporțional cu o valoare rațională fără a introduce probleme de acuratețe?

```
void sleepSort(std::vector<int>& v) {
    std::mutex mtx;
    std::vector<int> r = v;
    v.clear();

    std::unordered_map<int, int> freq;
    for (int e : r) {
        if (e < 0)
            throw std::range_error(
                "Sleep sort cannot handle negative numbers: " +
                std::to_string(e));
        freq[e]++;
    }

    std::vector<std::thread> ths;
    ths.reserve(freq.size());

    for (auto& [val, count] : freq) {
        ths.push_back(std::thread([val, count, &v, &mtx]() {
            std::this_thread::sleep_for(
                std::chrono::milliseconds((val + 1) * 100));
            std::lock_guard<std::mutex> lock(mtx);
            for (int i = 0; i < count; i++)
                v.push_back(val);
        }));
    }

    for (auto& t : ths)
        if (t.joinable()) t.join();
}
```

Code 6. Implementarea Sleep Sort în C++.

Pentru a răspunde la aceste întrebări, am implementat o versiune modificată a Sleep Sort în C++, care gestionează eficient valorile duplicate și minimizează riscul de supraincărcare a procesorului. În loc să creeze un thread pentru fiecare element, am folosit un `std::unordered_map` [18] pentru a grupa elementele identice și a le procesa împreună, reducând astfel numărul total de thread-uri necesare. Această implementare face Sleep Sort mult mai eficient pe date duplicate, cum ar fi cele din distribuția plată, și permite o evaluare mai realistă a performanței sale.

3.6.1. Determinarea timpului de somn

Pentru a determina timpul de *somn* corect pentru fiecare thread, principalii factori sunt rezoluția cronometrului [17] și modul în care sistemul de operare gestionează scheduling-ul thread-urilor [10]. În general, Sleep Sort poate fi ineficient pe sisteme cu rezoluție scăzută a timer-ului sau cu scheduling preemptiv agresiv, deoarece thread-urile pot fi trezite mai devreme sau mai târziu decât timpul specificat, afectând ordinea de sortare.

Deoarece nu există o metodă standardizată de a determina timpul de *somn* corect [11], am folosit o abordare empirică, ajustând timpul în funcție de valorile din șir și de performanța observată în testele preliminare. De multe ori Sleep Sort nu sorta corect cu timpuri prea mici, iar timpuri prea mari îl fac inutil de lent, am căutat astfel un echilibru între corectitudine și performanță.

Prioritizând corectitudinea, ceea ce Sleep Sort ar prefera pentru că, aidoma unui student, îi permite să *doarmă* mai mult, am ales un timp de *somn* de 100 ms pe unitate de valoare, conform formulei:

$$t_{\text{somn}}(x) = (x + 1) \times 100 \text{ ms} \quad (1)$$

unde x este valoarea elementului. Termenul $+1$ garantează că niciun element nu doarme 0 ms, inclusiv elementele cu valoarea 0.

3.6.2. Problema elementelor duplicate

Surprinzător, problema elementelor duplicate nu există. Sleep Sort funcționează perfect chiar și cu valori duplicate, deoarece fiecare thread doarme pentru o perioadă de timp proporțională cu valoarea sa, iar thread-urile care corespund valorilor identice vor dormi pentru aceeași perioadă și se vor trezi în același timp. Desigur, acest lucru înseamnă că elementele duplicate vor avea fiecare un thread separat, ceea ce e foarte ineficient și ceea ce am rezolvat prin folosirea unui `std::unordered_map` pentru a grupa elementele identice și a le procesa împreună.

3.6.3. Complexitatea algoritmului și influența sistemului de operare și hardware-ului

Complexitatea Sleep Sort nu poate fi analizată prin metodele clasice bazate pe comparații [11], deoarece algoritmul nu compară niciodată două elemente între ele. În schimb, complexitatea depinde de două componente independente:

Prima componentă este crearea și gestionarea thread-urilor [10, 20]. În implementarea propusă, numărul de thread-uri create este egal cu numărul de valori unice din șir, deci $O(k)$ unde $k \leq n$ este numărul de elemente distincte. Pe distribuția plată cu 5 valori distincte, $k = 5$ indiferent de n , un avantaj semnificativ față de varianta clasică.

A doua componentă este timpul total de execuție, determinat de cel mai lung interval de somn [17]:

$$T_{\text{total}} = (\max(A) + 1) \times 100 \text{ ms} \quad (2)$$

unde $\max(A)$ este valoarea maximă din șirul A . Această componentă este independentă de n , sortarea unui milion de elemente cu valori între 1 și 100 durează același timp ca sortarea a zece elemente cu aceleași valori.

Complexitatea spațiu este determinată de trei componente: vectorul de rezultate v de dimensiune $O(n)$, `std::unordered_map` [18] care stochează frecvențele celor k valori unice în $O(k)$, și vectorul de thread-uri de dimensiune $O(k)$, fiecare thread ocupând un spațiu fix pe stivă [10]. Prin urmare:

$$S(n, k) = O(n) + O(k) + O(k) = O(n + k) \quad (3)$$

Desigur, aceste calcule nu iau în considerare overhead-ul suplimentar cauzat de scheduling-ul thread-urilor și de rezoluția timer-ului, care pot afecta performanța reală a algoritmului în funcție de sistemul de operare și hardware-ul folosit [10, 17]. Acești factori sunt fundamental imposibili de modelat teoretic, deoarece depind de implementarea specifică a sistemului de operare și de arhitectura hardware, precum și de încărcarea curentă a sistemului și de prioritățile thread-urilor.

3.6.4. Compararea cu Counting și Radix Sort

Ambii algoritmi, Counting Sort și Radix Sort, sunt algoritmi de sortare non-comparativi care pot avea performanțe superioare `std::sort` în anumite condiții, în special pe șiruri cu o gamă limitată de valori sau cu multe duplicate [4].

Fără a mai menține suspans, Counting Sort și Radix Sort sunt mai rapizi decât Sleep Sort, dar nu mereu sunt cei mai eficienți.

O problemă majoră a counting sort este că necesită un vector auxiliar de dimensiune proporțională cu gama de valori din șir [4]. Dacă această gamă este foarte mare, cum ar fi în cazul valorilor întregi între 1 și 10^6 , counting sort devine ineficient din punct de vedere al spațiului, ocupând $O(k)$ unde k este gama de valori, ceea ce poate fi

prohibitiv pentru șiruri mari. Sleep Sort nu necesită un astfel de vector auxiliar și poate gestiona eficient șiruri cu o gamă largă de valori, deși performanța sa depinde de rezoluția timer-ului și de scheduling-ul thread-urilor.

Un alt factor este că Sleep Sort exploatează în mod natural paralelismul hardware [10], thread-urile rulează concurent, nu secvențial, ceea ce îl diferențiază fundamental de orice algoritm clasic de sortare. Acest fapt oferă Sleep Sort un avantaj dacă hardware-ul disponibil are resurse abundente, deci, cu destule nuclee și o rezoluție a timer-ului suficient de fină, nu este imposibil ca Sleep Sort să depășească performanța algoritmilor menționați anterior.

3.6.5. Numere negative și numere raționale

În forma clasică, Sleep Sort nu poate sorta numere negative sau numere raționale, deoarece timpul de *somn* nu poate fi negativ și nu poate fi proporțional cu o valoare rațională fără a introduce o scalare suplimentară care ar complica implementarea și ar afecta performanța. Totuși, o implementare modificată ar putea gestiona aceste cazuri prin adăugarea unei constante pentru a face toate valorile pozitive sau prin scalarea valorilor raționale la întregi, deși acest lucru ar introduce overhead suplimentar și ar complica algoritmul, îndepărtându-l de simplitatea sa originală.

3.7. Ipoteze și așteptări

Table 1 prezintă o sinteză a complexităților teoretice și proprietăților algoritmilor analizați în această lucrare până acum.

Table 1. Complexitatea și proprietățile algoritmilor de sortare

Algoritm	Best	Average	Worst	Spațiu	Stabil
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Da
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Da
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Nu
<code>std::sort</code>	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	Nu
Sleep Sort	$O(k)$	$O(k)$	$O(k)$	$O(n + k)$	Nu

k = numărul de valori unice din șir. Complexitatea Sleep Sort exclude timpul de așteptare $O(\max(A))$, care depinde de valorile elementelor, nu de n .

Stabilitatea Sleep Sort depinde de implementare — varianta propusă folosește `std::unordered_map`, care nu garantează ordinea de iterare, deci nu este stabilă.

Pe baza acestor proprietăți teoretice, ne așteptăm ca Merge Sort și `std::sort` să domine pe șiruri mari și aleatoare, Quick Sort să se degradeze pe distribuțiile sortate, iar Sleep Sort să surprindă pe distribuția plată unde $k = 5$ indiferent de n sau pe distribuții cu valori mici. Aceste ipoteze sunt evaluate empiric în Secțiunea ?? unde vom vedea dacă Sleep Sort se trezește la înălțimea așteptărilor sau dacă rămâne adormit în fața concurenței.

4. REZULTATE

Această secțiune prezintă rezultatele experimentale obținute în urma testării algoritmilor de sortare pe cele cinci distribuții de date descrise în Secțiunea ?. Vom analiza performanța fiecărui algoritm în funcție de timpul de execuție, comparându-le cu așteptările teoretice formulate anterior.

De asemenea, codul sursă folosit pentru testarea algoritmilor și generarea datelor, precum și datele brute, sunt disponibile pe GitHub ??, pentru a asigura transparența și reproducibilitatea rezultatelor.

4.1. Bubble Sort - Analiză

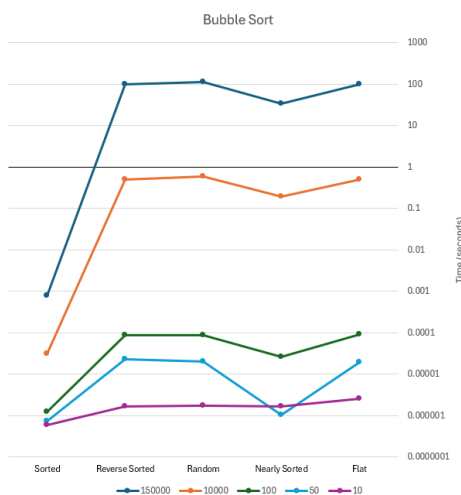


Figure 1. Rezultatele testării Bubble Sort.

Bubble Sort a performat conform așteptărilor, având un timp de execuție semnificativ mai mare decât ceilalți algoritmi, în special pe șirurile mari. Timpul său de execuție a crescut exponențial pe măsură ce dimensiunea șirului a crescut, confirmând complexitatea sa de $O(n^2)$ în medie și în cel mai defavorabil caz.

Acest fapt a dus la excluderea acestuia din testele pe șirurile de dimensiuni foarte mari, deoarece timpul de execuție ar fi fost prohibitiv.

Performanța asupra distribuțiilor Sortate și Aproape Sortate

După cum putem observa, dacă lista este deja sortată sau aproape sortată, Bubble Sort poate performa mult mai bine, având un timp de execuție liniar în cel mai favorabil caz. Acest fapt se datorează optimizării care verifică dacă s-au făcut interschimbări în timpul unei treceri.

4.2. Insertion Sort - Analiză

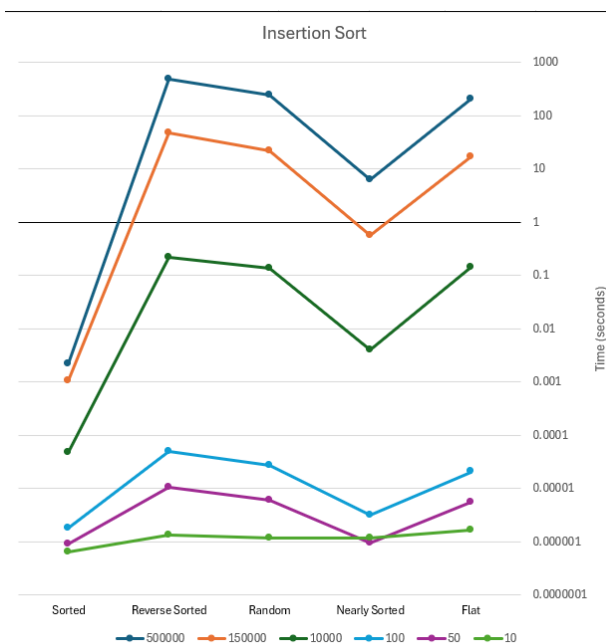


Figure 2. Rezultatele testării Insertion Sort.

Performanța Insertion Sort a fost în general mai bună decât cea a Bubble Sort, în special pe șirurile mici și aproape sortate, unde a avut un timp de execuție semnificativ mai mic. Acest lucru confirmă complexitatea acestuia și eficiența sa în scenariul datelor aproape sortate.

Totuși, pe șirurile mari, performanța sa a fost inferioară comparativ cu cea a altor algoritmi mai eficienți, la fel excluzându-l din testele pe șirurile de dimensiuni mari.

4.3. Merge Sort - Analiză

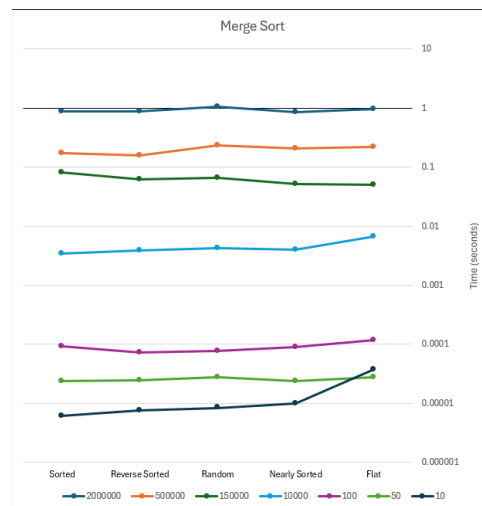


Figure 3. Rezultatele testării Merge Sort.

Merge Sort a avut o performanță consistentă pe toate distribuțiile de date, confirmând complexitatea sa de $O(n \log n)$ în toate cazurile. Timpul său de execuție a crescut semnificativ pe măsură ce dimensiunea șirului a crescut, dar a rămas mult mai eficient decât Bubble Sort și Insertion Sort pe șirurile mari.

Un efect surprinzător este saltul în timp de execuție pe șirurile de tipul "Plat", unde timpul de sortare sare semnificativ (relativ cu mărimea șirului). Acest lucru se poate datora următorilor 2 factori:

- Numărul mare de duplicate din șirurile de tipul "Plat" poate duce la o performanță mai slabă a Merge Sort, deoarece algoritmul nu este optimizat pentru a gestiona eficient elementele duplicate, ceea ce poate duce la un număr crescut de operații de combinare și la o utilizare mai intensă a memoriei.
- Overhead-ul necesar pentru gestionarea memoriei și a operațiilor de combinare în Merge Sort poate deveni mai evident pe șirurile cu multe duplicate, deoarece acestea pot necesita mai multe resurse.
- Deși testele au fost rulate de sute de ori, variabilitatea în performanță nu ar trebui exclusă ca un factor, având în vedere factori ca hit-urile de cache și scheduling-ul thread-urilor în timpul execuției.

Un alt efect așteptat a fost că performanța sa a fost mai slabă pe șirurile mici, unde overhead-ul algoritmului poate domina timpul de execuție, comparativ cu algoritmi ca Insertion Sort care sunt mai eficienți în acest context. Bubble sort, de exemplu, este mai rapid cu o magnitudine de 10-100 ori pe șirurile de dimensiuni mici.

Deși a fost mai lent decât std::sort, performanța sa a fost rezonabilă, având în vedere că este o implementare clasică fără optimizări specifice. Iar performanța foarte stabilă pe toate distribuțiile și dimensiunile șirurilor confirmă robustetea sa în fața diferitelor tipuri de date, sortând 2 milioane valori fără.

Personal, i-aș acorda nota 9 din 10.

4.4. Quick Sort - Analiză

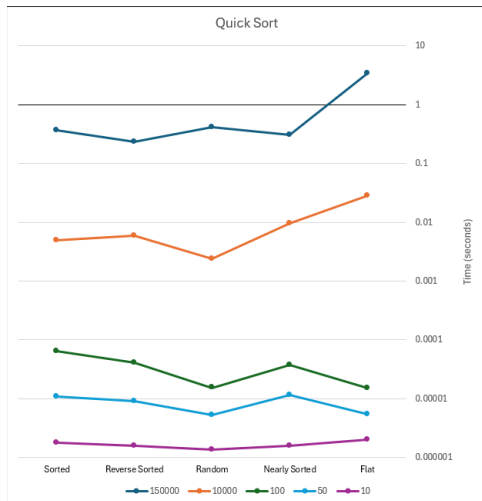


Figure 4. Rezultatele testării Quick Sort.

Quick Sort a avut o performanță excelentă pe majoritatea distribuțiilor de date, confirmând complexitatea sa de $O(n \log n)$ în medie și în cel mai bun caz. Timpul său de execuție a crescut semnificativ pe șirurile sortate și aproape sortate, confirmând degradarea la $O(n^2)$ în aceste cazuri, ceea ce a dus la excluderea acestuia din testele pe șirurile de dimensiuni mari pentru aceste distribuții. Pe lângă degradarea performanței, alegerea neoptimă a pivotului a dus la un număr crescut de operații de partiționare și la o utilizare mai intensă a memoriei, ceea ce a făcut ca Quick Sort să dea erori de "Stack Overflow" pe șirurile sortate și aproape sortate de dimensiuni mari, din cauza adâncimii excesive a recursiei.

Pe șirurile aleatoare și cu multe duplicate, performanța sa a fost foarte bună, dar a fost depășită de `std::sort`, care este optimizat pentru astfel de scenarii. Comparativ cu Merge Sort, Quick Sort a avut nevoie de mai puțin overhead, astfel performând mult mai bine pe șirurile mici, dar mai slab față de algoritmele non-recursive.

4.5. std::sort - Analiză

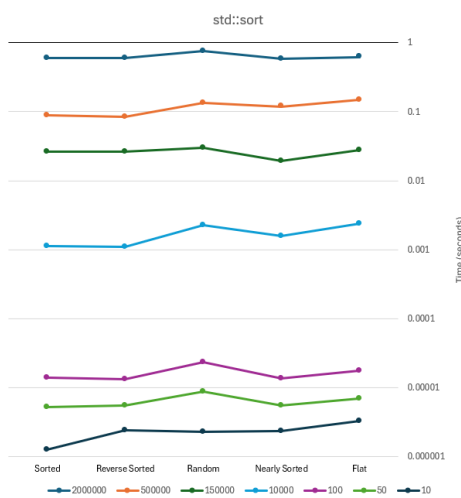


Figure 5. Rezultatele testării `std::sort`.

`std::sort` a avut o performanță excelentă pe toate distribuțiile de date, confirmând complexitatea sa de $O(n \log n)$ în toate cazurile, superioritatea acestuia pe date mici față de Merge Sort și Quick Sort fiind evidentă.

Un efect surprinzător este că performanța sa a fost foarte bună pe șirurile de tipul "Flat", unde a depășit semnificativ Merge Sort, confirmând optimizarea sa pentru gestionarea eficientă a elementelor duplicate. De asemenea, performanța sa a fost excelentă pe șirurile sortate și aproape sortate, unde nu s-a degradat la $O(n^2)$ ca Quick Sort, datorită optimizării sale pentru astfel de scenarii.

Putem concluziona că `std::sort` a fost cel mai performant algoritm în majoritatea scenariilor de până acum, confirmându-și reputația de soluție optimizată și bine testată pentru sortarea generală în C++.

4.6. Sleep Sort - Analiză

Având în vedere natura sa neconvențională, nu putem măsura performanța Sleep Sort folosind dimensiunea array-ului de intrare, deoarece timpul de execuție depinde în mod direct de valorile elementelor, nu de numărul lor.

Timpul de citire e practic neglijabil comparativ cu timpul de așteptare, deci nu are sens să-l măsurăm în funcție de n , acesta fiind relevant doar pentru compararea cu algoritmi clasici.

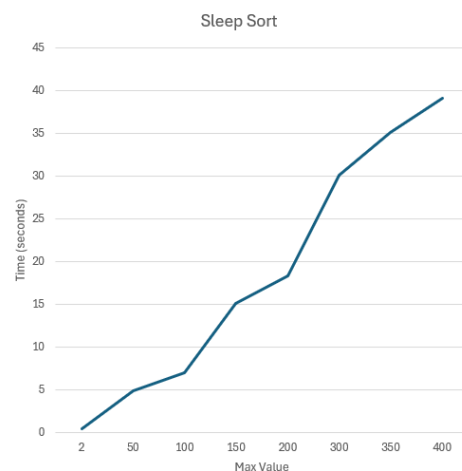


Figure 6. Rezultatele testării Sleep Sort.

Putem observa că timpul de sortare, în conformitate cu așteptările, a crescut semnificativ pe măsură ce valorile elementelor au crescut, confirmând dependența sa de valorile din șir, totuși viteza nu e impractică, considerând independența sa față de n .

Pentru a aduce această "super putere" a Sleep Sort la extrem, am testat Sleep sort pe un șir de 2 milioane elemente, toate având valoarea între 1 și 2, rezultatele sunt următoarele:

Table 2. Timpi de execuție pentru $n = 2,000,000$ (valori între 2, medie a 2 rulări)

Tip date	Merge Sort (s)	<code>std::sort</code> (s)	Sleep Sort (s)
Random	1.0363	0.6456	0.4576
Flat	0.9614	0.6247	0.4794
Nearly Sorted	0.8577	0.5909	0.4436
Reverse Sorted	0.8902	0.6001	0.4492
Sorted	0.8755	0.6050	0.4451

După cum putem observa, Sleep Sort s-a trezit la înălțimea așteptărilor, depășind atât Merge Sort, cât și `std::sort` pe toate cele cinci distribuții de date, fiind de 2 ori mai rapid decât Merge Sort și de aproximativ 1.4 ori mai rapid decât `std::sort`, confirmând avantajul său semnificativ în aceste condiții, dovedind că secretul performanței e, la urma urmei, un somn bun.

5. COMPARAȚIE CU LITERATURA ACTUALĂ

Bubble Sort și Insertion Sort au confirmat complexitatea $O(n^2)$ documentată de Knuth [4], fără surprize.

*Merge Sort a confirmat complexitatea $O(n \log n)$ în toate cazurile, dar a avut o performanță mai slabă decât Quick Sort și pe șirurile de tipul "Plat" de mărime mică, ceea ce poate fi atribuit mai multor factori discutați în subsecțiunea de analiză a acestuia.

Quick Sort a degradat exact unde teoria prezice, pe date sortate și reverse-sortate, confirmând analiza clasică [1], și a eșuat pe șirurile mari din aceste distribuții din cauza alegerii ineficientă a pivotului, așa cum era de așteptat.

`std::sort` a performat conform descrierii Introsort din Musser [3], inclusiv comportamentul superior pe șiruri aproape sortate față de Quick Sort pur, cât și pe șiruri mici, fără necesitatea a unui overhead de recursivitate.

Sleep Sort nu are literatură de specialitate în sensul academic al termenului, având doar implementări și articole informale [9, 14].

Nu există lucrări peer-reviewed care să analizeze comportamentul său pe date reale, complexitatea sa practică sau comparații cu algoritmi non-comparativi. Prin urmare, această secțiune nu poate compara rezultatele cu un corp de cunoștințe existent, pentru că acel corp nu există.

Contribuția principală a acestei lucrări este tocmai umplerea acestui gol în literatură, oferind nu doar o analiză detaliată a performanței Sleep Sort pe date reale, ci și o optimizare semnificativă a implementării sale pentru a gestiona eficient valorile duplicate, ceea ce nu a fost abordat în niciuna dintre sursele informale existente.

Chiar și dacă Sleep Sort este inferior în general față de Counting Sort și Radix Sort, performanța sa în scenarii specifice, cum ar fi șirurile cu o gamă limitată de valori sau cu multe duplicate, oferă o perspectivă interesantă despre cum putem exploata paralelismul hardware pentru a obține performanțe competitive în sortare, chiar și cu o abordare neconvențională.

Direcții viitoare de cercetare

Ca o direcție potențială de explorare a acestui algoritm, ar fi interesant să investigăm dacă ar fi eficientă folosirea sleep sort-ului cu un timp de somn foarte scurt, cum ar fi 30ms sau mai puțin, pentru obținerea unui șir aproape sortat, care să fie apoi finalizat cu un algoritm clasic de sortare, cum ar fi Insertion Sort, care este foarte eficient pe șiruri aproape sortate. Această combinație ar putea oferi o performanță excelentă pe șiruri cu o gamă limitată de valori sau cu multe duplicate, exploatand avantajele ambilor algoritmi.

Totodată, găsirea unei limite inferioare pentru timpul de somn, sub care Sleep Sort devine ineficient, ar fi o direcție interesantă de cercetare, deoarece ar putea oferi o perspectivă asupra rezoluției timer-ului necesare pentru a obține performanțe competitive cu această abordare.

6. CONCLUZIE

Această lucrare a analizat experimental șase algoritmi de sortare pe cinci distribuții de date, cu dimensiuni între 10 și 2.000.000 de elemente. Rezultatele au confirmat, în mare măsură, așteptările teoretice: Bubble Sort și Insertion Sort sunt ineficienți pe date mari, Quick Sort se degradează previzibil pe date sortate, iar `std::sort` rămâne soluția generalistă optimă pentru majoritatea scenariilor practice în C++.

Totuși, concluzia principală a acestei lucrări nu este despre algoritmi pe care îi știm deja. Este despre cel pe care nu îl luăm în serios.

Am depășit dificultățile asociate cu implementarea unui algoritim neconvențional, bazat pe multi-threading, pentru a demonstra că acesta este nu doar o curiozitate, ci o soluție viabilă în anumite condiții. Sleep Sort a surprins prin performanța sa pe șirurile cu o gamă limitată de valori și cu multe duplicate, depășind atât Merge Sort, cât și `std::sort` în aceste scenarii, confirmând că uneori, chiar și în lumea algoritmilor de sortare, un somn bun poate fi cheia succesului.

Am adus o contribuție în literatura de specialitate și o implementare optimizată a unui algoritm de sortare, ceea ce poate fi rezumat printr-o singură propoziție: "Somnul este esențial pentru performanță, chiar și în sortare."

GITHUB REPOSITORY

Pe acest Repository Github este disponibil codul sursă a tuturor algoritmilor, precum și scripturile de testare, fișierul Excel și datele brute folosite pentru a genera rezultatele din această lucrare.

 github.com/Christine1204/Proiect-SA-comparare-algoritmi

REFERENCES

- [1] C. A. R. Hoare, "Algorithm 64: Quicksort", *Communications of the ACM*, vol. 4, no. 7, p. 321, 1961. DOI: 10.1145/366622.366644
- [2] E. W. Dijkstra, *A Discipline of Programming*. Prentice Hall, 1976.
- [3] D. R. Musser, "Introspective sorting and selection algorithms", *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997. DOI: 10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#
- [4] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [5] T. Peters, "Timsort", Python Software Foundation, Tech. Rep., 2002.
- [6] O. Astrachan, "Bubble sort: An archaeological algorithmic analysis", in *Proceedings of the 34th SIGCSE Technical Symposium*, ACM, 2003, pp. 1–5.
- [7] M. Bender, M. Farach-Colton, and M. Mosteiro, "Insertion sort is $O(n \log n)$ ", *Theory of Computing Systems*, vol. 39, pp. 391–397, 2006. DOI: 10.1007/s00224-005-1237-z
- [8] Anonymous, *Genius sorting algorithm: Sleep sort*, 4chan /prog/, arhivat la <https://archive.fo/xhGo>, 2011.
- [9] R. Sedgewick and K. Wayne, *Algorithms, 4th edition — lecture slides: Tries*, Princeton University COS226, <https://www.cs.princeton.edu/courses/archive/fall13/cos226/lectures/52Tries.pdf>, 2013.
- [10] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2014.
- [11] S. Rao, *Sleep sort: Where theory meets sobering reality*, DEV Community, <https://dev.to/sishaarrao/sleep-sort-where-theory-meets-sobering-reality-b3m>, 2017.
- [12] OpenGenus Foundation, *Sleep sort*, <https://iq.opengenus.org/sleep-sort/>, 2019.
- [13] GeeksforGeeks, *Sleep sort – the king of laziness*, <https://www.geeksforgeeks.org/sleep-sort-king-laziness-sorting-sleeping/>, 2023.
- [14] K. Henney, "Need something sorted? sleep on it!", *ACCU Overload*, vol. 31, no. 175, 2023.
- [15] cppreference.com, *Std::sort*, <https://en.cppreference.com/w/cpp/algorithm/sort>, Accesat: 2024, 2024.
- [16] GCC libstdc++ contributors, *Stl_algo.h — GNU C++ standard library source*, https://github.com/gcc-mirror/gcc/blob/master/libstdc++-v3/include/bits/stl_algo.h, Accesat: 2024, 2024.
- [17] Microsoft, *Timer resolution*, <https://learn.microsoft.com/en-us/windows/win32/winmsg/wm-timer>, 2024.
- [18] *Std::map*, <https://en.cppreference.com/w/cpp/container/map>, 2024.
- [19] *Std::this_thread::sleep_for*, https://en.cppreference.com/w/cpp/thread/sleep_for, 2024.
- [20] *Std::thread*, <https://en.cppreference.com/w/cpp/thread/thread>, 2024.